



## گزارش جامع پروژه شماره ۳

طراحی و پیاده سازی سیستم طبقه بندی تصاویر زبان اشاره و مترجم  
بلادرنگ به گفتار

پروژه هوش مصنوعی

دکتر تن قطاری

۴۰۳۱۰۵۸۰۹ | فاطمه بهزادی دیل

۴۰۳۱۱۰۲۴۱ | فاطمه دمیرچی

[github.com/FDamirchi/sign-language-translator](https://github.com/FDamirchi/sign-language-translator)

نام درس

استاد

دانشجوی اول

دانشجوی دوم

مخزن پروژه

## چکیده

این پروژه یک سامانه انتهابه‌انتهای تشخیص حروف زبان اشاره آمریکایی از تصویر، تجمیع خروجی به متن و تبدیل متن نهایی به گفتار را پیاده‌سازی می‌کند. داده مورد استفاده شامل ۸۷۰۰۰ تصویر آموزشی در ۲۹ کلاس است: حروف A-Z و سه فرمان `del`، `nothing` و `space`. تحلیل اکتشافی نشان می‌دهد تمام کلاس‌های مجموعه آموزشی دقیقاً ۳۰۰۰ تصویر دارند، تصاویر همگی RGB و با اندازه ثابت  $200 \times 200$  پیکسل هستند و در نتیجه داده از نظر تعداد نمونه کاملاً متوازن است.

در فاز مدل‌سازی، یک شبکه عصبی پیچشی چهاربخشی با ۱۹۲۷۸۱۷۳ پارامتر طراحی و با چارچوب PyTorch آموزش داده شد. داده‌ها با نسبت‌های ۷۵/۱۵/۱۰ به آموزش، اعتبارسنجی و آزمون تقسیم شدند. مدل با بهینه‌ساز Adam، نرخ یادگیری ۰.۰۰۱، اندازه دسته ۶۴ و به مدت ۱۰ دوره آموزش دید. بهترین دقت اعتبارسنجی در دوره نهم برابر ۹۹.۲۳٪ و دقت آزمون نهایی برابر ۹۹.۳۴٪ گزارش شد.

در فاز بلادرنگ، یک معماری ماژولار برای دریافت تصویر وب‌کم، استخراج ناحیه دست، همسان‌سازی پیش‌پردازش با مدل، استنتاج روی CPU/GPU، تثبیت زمانی پیش‌بینی، مدیریت فرمان‌های ویرایشی، تشخیص پایان رشته و پخش صوت با pytsx3 ایجاد شد. علامت تنها زمانی ثبت می‌شود که با اطمینان حداقل ۸۰٪ برای ثانیه ۲ پایدار بماند؛ کلاس NOTHING پس از ثانیه ۰.۴ قفل علامت قبلی را آزاد و پس از ثانیه ۵ متن موجود را نهایی می‌کند. کد با تست‌های واحد برای اجزای اصلی و کنترل کیفیت با pytest و Ruff همراه شده است.

# فهرست مطالب

|    |                                              |
|----|----------------------------------------------|
| آ  | چکیده                                        |
| ۱  | ۱ تعریف مسئله، اهداف و معیارهای تحویل        |
| ۱  | ۱.۱ بیان مسئله                               |
| ۱  | ۲.۱ اهداف فنی                                |
| ۲  | ۳.۱ تطبیق الزامات صورت پروژه با خروجی نهایی  |
| ۳  | ۲ فاز اول: تحلیل اکتشافی و پیش‌پردازش داده   |
| ۳  | ۱.۲ ساختار مجموعه داده                       |
| ۴  | ۲.۲ تصویرسازی نمونه‌ها                       |
| ۵  | ۳.۲ ویژگی‌های تصاویر                         |
| ۶  | ۴.۲ تقسیم‌بندی داده                          |
| ۷  | ۵.۲ نرمال‌سازی داده‌ها                       |
| ۹  | ۳ فاز دوم: طراحی و آموزش شبکه عصبی پیچشی     |
| ۹  | ۱.۳ معماری مدل                               |
| ۱۰ | ۱.۱.۳ دلایل انتخاب اجزای معماری              |
| ۱۱ | ۲.۳ افزایش داده و کنترل بیش‌برازش            |
| ۱۲ | ۱.۲.۳ تکنیک‌های استفاده‌شده برای بهبود تعمیم |
| ۱۲ | ۳.۳ پارامترهای آموزش                         |
| ۱۳ | ۱.۳.۳ انتخاب بهینه‌ساز                       |
| ۱۴ | ۴.۳ روند آموزش                               |

|     |                                              |    |
|-----|----------------------------------------------|----|
| ۵.۳ | ارزیابی نهایی                                | ۱۵ |
| ۴   | فاز سوم: سامانه بلادرنگ ترجمه به متن و گفتار | ۱۸ |
| ۱.۴ | معماری کلان                                  | ۱۸ |
| ۲.۴ | ساختار ماژول‌ها                              | ۱۸ |
| ۳.۴ | دریافت تصویر و ناحیه مورد توجه               | ۱۹ |
| ۴.۴ | همسان‌سازی پیش‌پردازش آموزش و اجرا           | ۱۹ |
| ۵.۴ | بارگذاری مدل و استنتاج                       | ۲۰ |
| ۶.۴ | منطق تثبیت زمانی                             | ۲۰ |
| ۷.۴ | تفسیر برچسب‌ها و بافر متن                    | ۲۱ |
| ۸.۴ | تشخیص پایان عبارت و تبدیل به گفتار           | ۲۱ |
| ۹.۴ | رابط تصویری                                  | ۲۲ |
| ۵   | اعتبارسنجی نرم‌افزار و کیفیت پیاده‌سازی      | ۲۳ |
| ۱.۵ | مجموعه تست‌ها                                | ۲۳ |
| ۲.۵ | مدیریت خطا                                   | ۲۴ |
| ۳.۵ | قابلیت تعویض اجزا                            | ۲۴ |
| ۶   | تحلیل محدودیت‌ها و پیشنهادهای بهبود          | ۲۵ |
| ۱.۶ | شکاف دامنه میان دیتاست و وب‌کم               | ۲۵ |
| ۲.۶ | حروف و حرکات پویا                            | ۲۵ |
| ۷   | راهنمای اجرا و ساختار تحویل                  | ۲۶ |
| ۱.۷ | نصب                                          | ۲۶ |
| ۲.۷ | اجرای کنترل کیفیت                            | ۲۶ |
| ۳.۷ | اجرای سامانه                                 | ۲۶ |
| ۴.۷ | ساختار فایل نهایی تحویل                      | ۲۷ |
| ۸   | نتیجه‌گیری                                   | ۲۸ |

۲۹

آ تنظیمات نهایی سامانه

۳۰

ب قرارداد وزن‌های مدل

---

## فهرست تصاویر

|    |                                                                     |     |
|----|---------------------------------------------------------------------|-----|
| ۴  | توزیع کلاس‌ها در پوشه آموزشی                                        | ۱.۲ |
| ۵  | نمونه‌ای از هر کلاس مجموعه آموزشی؛ تمام تصاویر $200 \times 200$ RGB | ۲.۲ |
| ۷  | نسبت بخش‌های آموزش، اعتبارسنجی و آزمون                              | ۳.۲ |
| ۹  | معماری شبکه SignLanguageCNN                                         | ۱.۳ |
| ۱۴ | دقت آموزش و اعتبارسنجی                                              | ۲.۳ |
| ۱۶ | ماتریس درهم‌ریختگی روی مجموعه آزمون                                 | ۳.۳ |
| ۱۸ | زنجیره پردازش بلادرنگ از وب‌کم تا گفتار                             | ۱.۴ |
| ۲۱ | ماشین حالت رمزگشایی زمانی                                           | ۲.۴ |

---

## فهرست جداول

|     |                                              |    |
|-----|----------------------------------------------|----|
| ۱.۱ | پوشش الزامات اصلی پروژه                      | ۲  |
| ۱.۲ | خلاصه ویژگی‌های تصویری داده                  | ۶  |
| ۲.۲ | تقسیم‌بندی نهایی داده برای مدل‌سازی          | ۶  |
| ۱.۳ | جزئیات لایه‌ها و تعداد پارامترهای قابل آموزش | ۱۰ |
| ۲.۳ | ابزارهای آموزش                               | ۱۲ |
| ۳.۳ | دقت ثبت‌شده در هر دوره                       | ۱۴ |
| ۴.۳ | نتایج نهایی مدل                              | ۱۵ |
| ۵.۳ | پنج جفت کلاس با بیشترین خطا                  | ۱۶ |
| ۱.۴ | رفتار هر نوع برچسب پذیرفته‌شده               | ۲۱ |
| ۱.۵ | حوزه‌های تحت آزمون واحد                      | ۲۳ |
| ۱.آ | مقادیر پیش‌فرض تنظیمات اجرایی                | ۲۹ |

# تعریف مسئله، اهداف و معیارهای تحویل

## ۱.۱ بیان مسئله

هدف پروژه توسعه یک سیستم کامل هوش مصنوعی از مرحله شناخت داده خام تا استقرار در یک محیط کاربردی بلادرنگ است. ورودی سیستم فریم‌های دوربین و خروجی آن متن حروف‌چینی‌شده و صوت متن نهایی است. مسئله اصلی را می‌توان به سه زیرمسئله تقسیم کرد:

۱. تحلیل و استانداردسازی تصاویر کلاس‌بندی‌شده زبان اشاره؛
۲. آموزش یک طبقه‌بند CNN برای تشخیص یکی از ۲۹ کلاس؛
۳. طراحی منطق زمانی برای تبدیل پیش‌بینی‌های فریم‌به‌فریم به رشته پایدار و قابل خواندن.

## ۲.۱ اهداف فنی

- ▶ بررسی توازن کلاس‌ها، ویژگی‌های تصویر و توزیع داده؛
- ▶ طراحی شبکه شامل لایه‌های Convolution، MaxPooling و Fully Connected؛
- ▶ کاهش بیش‌برازش با افزایش داده، Dropout و تنظیم نرخ یادگیری؛
- ▶ دریافت تصویر با OpenCV و محدودسازی ورودی مدل به یک ناحیه مربعی؛
- ▶ ثبت علامت تنها پس از تشخیص پایدار برای جلوگیری از نویز لحظه‌ای؛
- ▶ پشتیبانی از فرمان‌های DEL، SPACE و NOTHING؛
- ▶ تبدیل رشته نهایی به گفتار و آماده‌سازی سیستم برای عبارت بعدی؛
- ▶ جداسازی اجزا و فراهم کردن تست‌های واحد برای توسعه و نگهداری مطمئن.



## ۳.۱ تطبیق الزامات صورت پروژه با خروجی نهایی

جدول ۱.۱: پوشش الزامات اصلی پروژه

| فاز   | الزام                                                           | وضعیت     |
|-------|-----------------------------------------------------------------|-----------|
| فاز ۱ | شمارش نمونه‌ها و بررسی توازن ۲۹ کلاس                            | انجام‌شده |
| فاز ۱ | نمایش نمونه تصادفی از هر کلاس در شبکه تصویری                    | انجام‌شده |
| فاز ۱ | تحلیل ابعاد، کانال رنگ و بازه پیکسل‌ها                          | انجام‌شده |
| فاز ۱ | تقسیم داده به آموزش، اعتبارسنجی و آزمون                         | انجام‌شده |
| فاز ۲ | طراحی و آموزش CNN و شرح معماری                                  | انجام‌شده |
| فاز ۲ | بررسی اثر پارامترهایی نظیر نرخ یادگیری، اندازه دسته و بهینه‌ساز | انجام‌شده |
| فاز ۲ | ارزیابی روی داده آزمون و گزارش دقت                              | انجام‌شده |
| فاز ۲ | نمودار Accuracy و Loss و ماتریس درهم‌ریختگی                     | انجام‌شده |
| فاز ۳ | دریافت تصویر وب‌کم، کادر دست و ارسال ROI به مدل                 | انجام‌شده |
| فاز ۳ | تثبیت علامت برای ۲ ثانیه با اطمینان بالا                        | انجام‌شده |
| فاز ۳ | نهایی‌سازی بعد از ۵ ثانیه NOTHING                               | انجام‌شده |
| فاز ۳ | تبدیل متن به گفتار و پاک‌کردن رشته                              | انجام‌شده |

# فاز اول: تحلیل اکتشافی و پیش‌پردازش داده

## ۱.۲ ساختار مجموعه داده

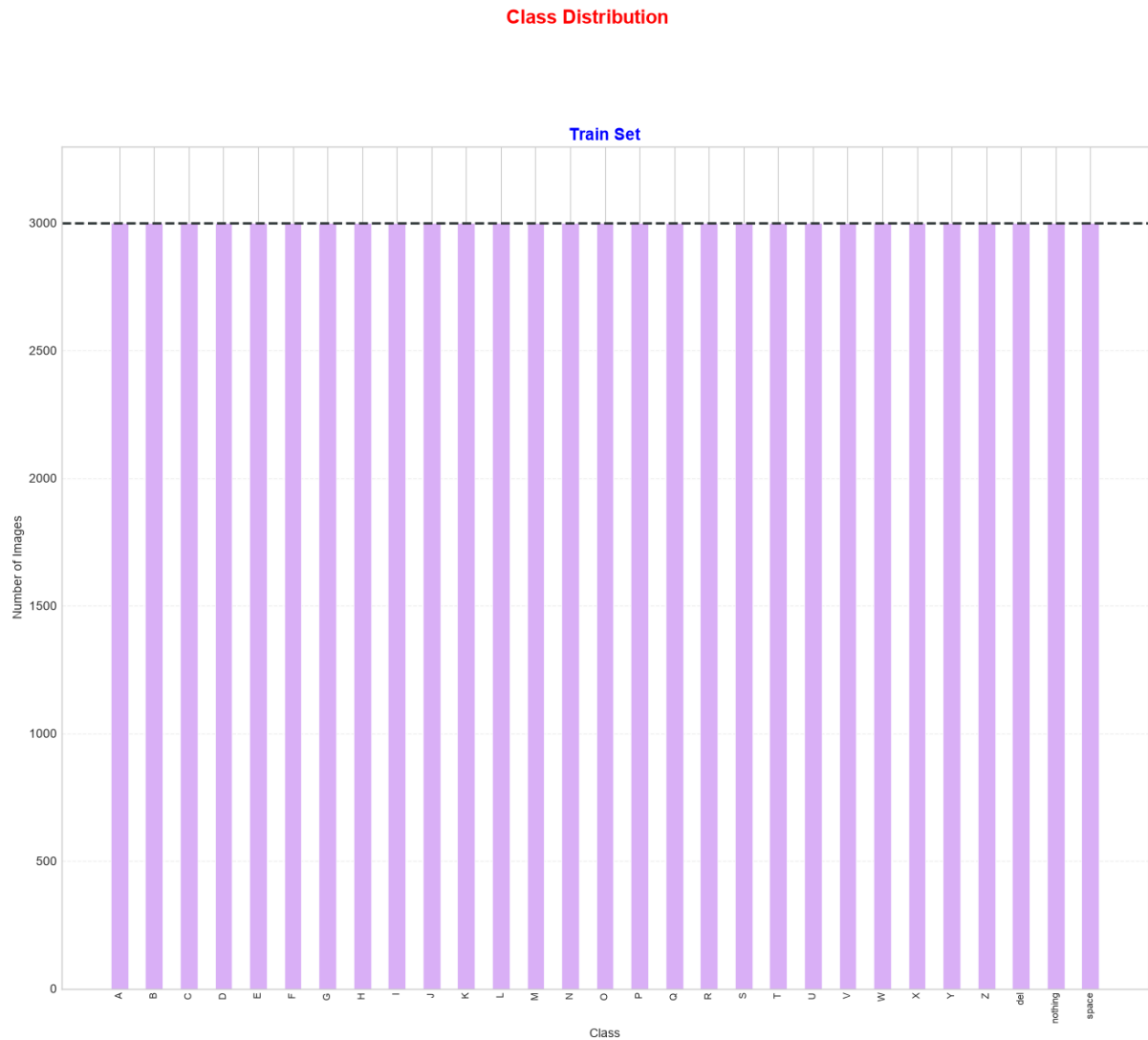
مجموعه آموزشی شامل ۲۹ پوشه کلاس است. ۲۶ کلاس نخست حروف الفبای انگلیسی و سه کلاس آخر برای کنترل رشته در محیط بلادرنگ در نظر گرفته شده‌اند:

A, B, C, ..., Z, del, nothing, space

هر کلاس دقیقاً ۳,۰۰۰ تصویر دارد؛ بنابراین تعداد کل تصاویر آموزشی برابر است با:

$$29 \times 3000 = 87000$$

در پوشه آزمون اولیه دیتاست تنها ۲۸ تصویر، هر کلاس یک تصویر، وجود داشته و کلاس del غایب بوده است. این مجموعه آزمون کوچک برای ارزیابی آماری مدل کافی نبود؛ از این رو برای فاز آموزش یک تقسیم‌بندی جدید از تصاویر اصلی ایجاد شد.

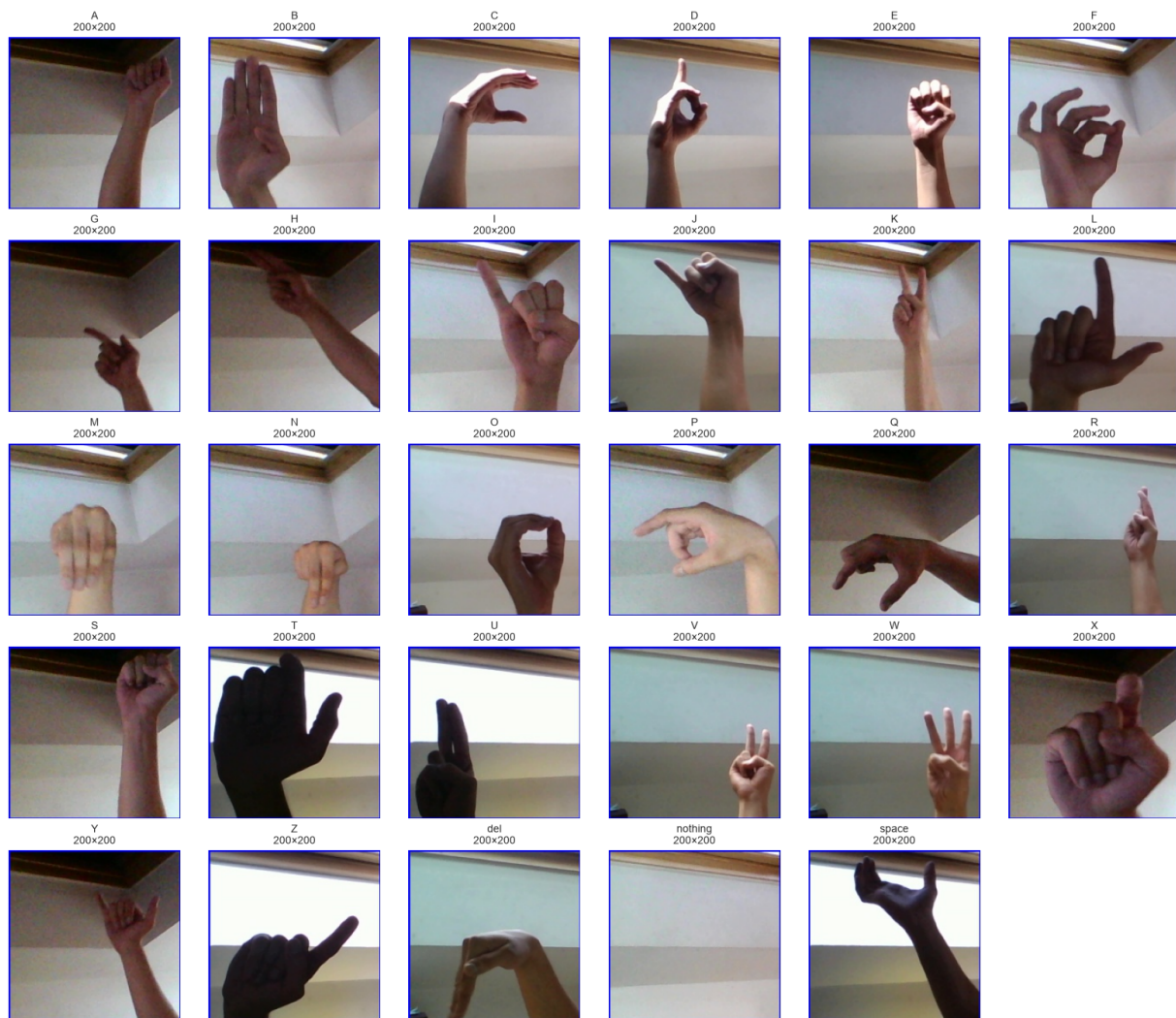


شکل ۱.۲: توزیع کلاس‌ها در پوشه آموزشی

## ۲.۲ تصویرسازی نمونه‌ها

شکل ۲.۲ یک تصویر تصادفی از هر یک از ۲۹ کلاس را نشان می‌دهد. تصاویر شامل دست، مچ و بخشی از ساعد هستند و پس‌زمینه ثابت و قابل‌توجهی نیز در آن‌ها دیده می‌شود. این ویژگی در زمان استقرار مهم است؛ زیرا یک شبکه کانولوشنی ممکن است علاوه بر شکل دست، از نور، زاویه و الگوی پس‌زمینه نیز سرنخ بگیرد.

Sample Image from Each Class In Train Set



شکل ۲.۲: نمونه‌ای از هر کلاس مجموعه آموزشی؛ تمام تصاویر 200×200 RGB

## ۳.۲ ویژگی‌های تصاویر

تحلیل تمام 87000 تصویر نتایج زیر را نشان داد:

جدول ۱.۲: خلاصه ویژگی‌های تصویر داده

| ویژگی                 | نتیجه                                 |
|-----------------------|---------------------------------------|
| حالت رنگ              | همه تصاویر RGB                        |
| عرض                   | همگی 200 پیکسل                        |
| ارتفاع                | همگی 200 پیکسل                        |
| نسبت تصویر            | ثابت و برابر 1.00                     |
| بازه ذخیره‌سازی پیکسل | اعداد صحیح 0-255                      |
| توازن کلاس‌ها         | انحراف معیار تعداد نمونه‌ها برابر صفر |

همان‌طور که در جدول بالا مشاهده می‌شود، تمام تصاویر موجود در مجموعه داده از ابعاد یکسان و حالت رنگی RGB برخوردار هستند که این امر فرآیند پیش‌پردازش را به‌طور قابل‌توجهی ساده می‌سازد. همچنین داده‌ها از توازن کامل برخوردار بوده و هر یک از 29 کلاس دقیقاً شامل 3000 تصویر می‌باشند. این توازن کامل در کنار یکنواختی ابعاد تصاویر، مجموعه داده را به گزینه‌ای ایده‌آل برای آموزش مدل‌های یادگیری عمیق تبدیل کرده است و نیاز به تکنیک‌های متعادل‌سازی یا تغییر اندازه را به‌حداقل می‌رساند.

با وجود ثابت‌بودن اندازه داده، در هر دو مسیر آموزش و استنتاج عمل تغییر اندازه به  $200 \times 200$  تکرار شده است تا ورودی‌های خارج از دیتاست نیز به شکل مورد انتظار مدل تبدیل شوند.

## ۴.۲ تقسیم‌بندی داده

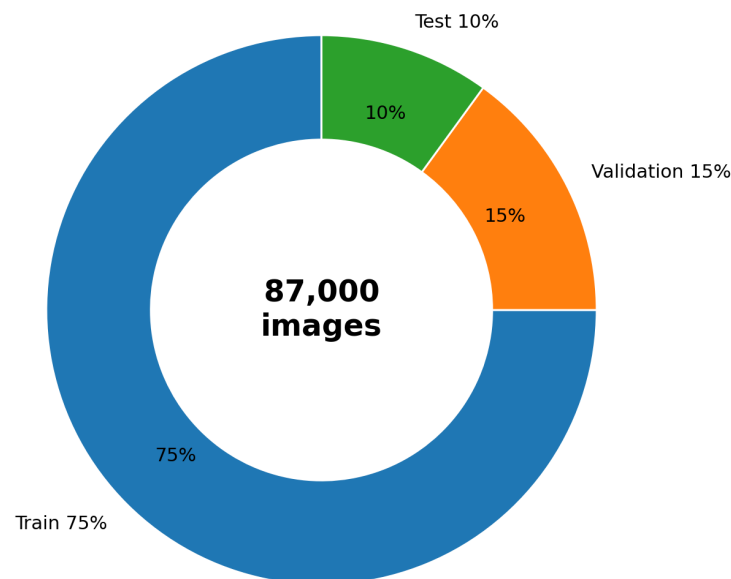
از کل 87000 تصویر، تقسیم زیر استفاده شده است:

جدول ۲.۲: تقسیم‌بندی نهایی داده برای مدل‌سازی

| بخش        | تعداد تصویر | نسبت |
|------------|-------------|------|
| Train      | 65250       | 75%  |
| Validation | 13050       | 15%  |
| Test       | 8700        | 10%  |
| جمع        | 87000       | 100% |

این تقسیم‌بندی بر اساس رویه‌های استاندارد در پروژه‌های یادگیری ماشین انتخاب شده است. نسبت 75 درصد برای داده‌های آموزش، حجم مناسبی از داده‌ها را برای یادگیری الگوها در اختیار مدل قرار می‌دهد. استفاده از 15 درصد داده‌ها برای اعتبارسنجی، امکان پایش عملکرد مدل در حین آموزش و تنظیم فرآیندها را بدون دخالت در فرآیند یادگیری فراهم می‌کند. همچنین در نظر گرفتن 10 درصد داده‌ها به‌عنوان مجموعه آزمون، ارزیابی نهایی

و بی‌طرفانه از عملکرد مدل روی داده‌های کاملاً دیده‌نشده را امکان‌پذیر می‌سازد. این نسبت‌ها به‌خوبی تعادل بین آموزش کافی، اعتبارسنجی دقیق و ارزیابی قابل‌اطمینان را برقرار می‌کنند.



شکل ۳.۲: نسبت بخش‌های آموزش، اعتبارسنجی و آزمون

## ۵.۲ نرمال‌سازی داده‌ها

پس از بررسی ساختار داده‌ها، مرحله نرمال‌سازی به‌عنوان یکی از مهم‌ترین مراحل پیش‌پردازش انجام شد. هدف از این مرحله، نگاشت مقادیر پیکسل‌ها به توزیعی با میانگین صفر و واریانس واحد است که همگرایی مدل را تسریع کرده و از افت عملکرد ناشی از تغییرات شدید مقادیر ورودی جلوگیری می‌کند.

برای هر کانال رنگی، میانگین و انحراف معیار روی تمام تصاویر مجموعه آموزش محاسبه شد:

$$\mu = (0.5188341, 0.4989899, 0.5144102),$$

$$\sigma = (0.2045820, 0.2336411, 0.2412035).$$

پس از تبدیل پیکسل‌ها به بازه  $[0,1]$  توسط تبدیل  $\text{ToTensor}()$ ، هر کانال به‌صورت جداگانه با استفاده از رابطه زیر استاندارد شد:

$$x'_c = \frac{x_c - \mu_c}{\sigma_c}$$

که در آن  $c \in \{R, G, B\}$  نشان‌دهنده کانال رنگی است.

آمار محاسبه‌شده در فایل `dataset_stats.pt` ذخیره شده و در فرآیند استنتاج بلادرنگ نیز از همین مقادیر استفاده می‌شود. این همسانی در نرمال‌سازی، از تغییر توزیع داده‌های ورودی میان فاز آموزش و فاز اجرا جلوگیری

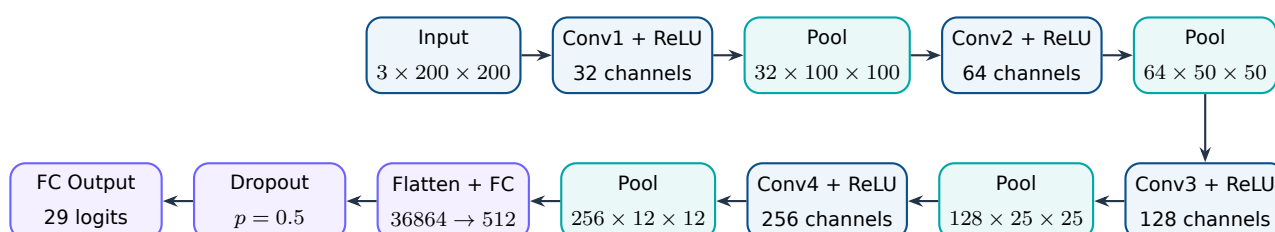
کرده و یکی از عوامل کلیدی در حفظ دقت مدل در محیط عملیاتی است.

لازم به ذکر است که استفاده از آمار اختصاصی مجموعه داده، نسبت به استفاده از میانگین و انحراف معیار پیش فرض ImageNet، عملکرد بهتری را در پی داشته است. دلیل این بهبود، نزدیکی بیشتر این آمار به توزیع واقعی داده‌های زبان اشاره است که از نظر رنگ‌بندی، بافت و نورپردازی با تصاویر طبیعی تفاوت دارد.

## فاز دوم: طراحی و آموزش شبکه عصبی پیچشی

### ۱.۳ معماری مدل

مدل SignLanguageCNN چهار بلوک کانولوشن و بیشینه‌گیری، یک لایه پنهان تمام‌متصل و یک لایه خروجی ۲۹ کلاسه دارد. در تمام لایه‌های کانولوشن از هسته  $3 \times 3$ ،  $\text{padding}=1$  و تابع فعال‌ساز ReLU استفاده شده است. هر بلوک با  $\text{MaxPool } 2 \times 2$  ابعاد مکانی را نصف می‌کند.



شکل ۱.۳: معماری شبکه SignLanguageCNN



جدول ۱.۳: جزئیات لایه‌ها و تعداد پارامترهای قابل آموزش

| لایه  | ورودی                      | خروجی پس از Pool           | پارامتر    | توضیح                                    |
|-------|----------------------------|----------------------------|------------|------------------------------------------|
| Conv1 | $3 \times 200 \times 200$  | $32 \times 100 \times 100$ | 896        | استخراج لبه‌ها و الگوهای ابتدایی         |
| Conv2 | $32 \times 100 \times 100$ | $64 \times 50 \times 50$   | 18,496     | ترکیب ویژگی‌های سطح پایین                |
| Conv3 | $64 \times 50 \times 50$   | $128 \times 25 \times 25$  | 73,856     | یادگیری الگوهای پیچیده‌تر شکل دست        |
| Conv4 | $128 \times 25 \times 25$  | $256 \times 12 \times 12$  | 295,168    | تولید نمایش فشرده سطح بالا               |
| FC1   | 36,864                     | 512                        | 18,874,880 | تبدیل ویژگی‌های مکانی به نمایش طبقه‌بندی |
| FC2   | 512                        | 29                         | 14,877     | تولید لاجیت هر کلاس                      |
| مجموع |                            |                            | 19,278,173 |                                          |

### ۱.۱.۳ دلایل انتخاب اجزای معماری

تعداد لایه‌های کانولوشن

تعداد 4 لایه کانولوشن بر اساس ویژگی‌های داده و آزمایش‌های اولیه انتخاب شده است. تصاویر حروف اشاره شامل الگوهای محدودی هستند و 4 لایه برای استخراج ویژگی‌های ساده تا پیچیده کافی است. همچنین با توجه به ابعاد  $200 \times 200$  پیکسل، پس از 4 بار بیشینه‌گیری، ابعاد به  $12 \times 12$  کاهش می‌یابد که حفظ اطلاعات مکانی را تضمین می‌کند. آزمایش‌های اولیه با 3 و 5 لایه نشان داد که 3 لایه دقت کافی ندارد و 5 لایه نیز بدون بهبود چشمگیر، تعداد پارامترها و خطر بیش‌برازش را افزایش می‌دهد. بنابراین 4 لایه به‌عنوان معماری نهایی انتخاب شده است.

هسته  $3 \times 3$  و  $\text{padding}=1$

هسته  $3 \times 3$  کوچک‌ترین هسته مؤثر برای تشخیص الگوهای محلی مانند لبه‌ها و بافت‌ها است و تعداد پارامتر کمی دارد.  $\text{padding}=1$  نیز باعث می‌شود ابعاد تصویر در طول کانولوشن حفظ شود و اطلاعات لبه‌ها از دست نرود. این ترکیب، تعادل مناسبی بین قدرت تشخیص و حجم محاسبات ایجاد می‌کند.

افزایش تدریجی تعداد کانال‌ها

تعداد کانال‌ها از 32 به 256 افزایش یافته است. لایه‌های اولیه برای تشخیص الگوهای ساده مانند لبه‌ها به کانال کمتری نیاز دارند، درحالی‌که لایه‌های عمیق‌تر برای ترکیب ویژگی‌ها و تشخیص الگوهای پیچیده‌تر مانند شکل انگشتان، به کانال بیشتری نیاز دارند. این افزایش تدریجی، تعادل بین قدرت تشخیص و حجم محاسبات را حفظ

می‌کند.

## لایه MaxPool 2×2

لایه MaxPool 2×2 ابعاد مکانی را نصف می‌کند و باعث کاهش حجم داده، افزایش مقاومت در برابر جابه‌جایی جزئی دست و انتخاب ویژگی‌های مهم می‌شود. این کار همچنین تعداد پارامترهای لایه‌های بعدی را کاهش داده و سرعت پردازش را افزایش می‌دهد.

## تابع فعال‌ساز ReLU

تابع ReLU به دلیل سادگی، سرعت بالا و کاهش مشکل محو شدن گرادیان در شبکه‌های عمیق انتخاب شده است. این تابع از نظر محاسباتی بسیار کارآمد است و در برابر اشباع شدن نورون‌ها مقاومت خوبی نشان می‌دهد.

## لایه FC1 با 512 نورون

512 نورون در لایه پنهان تمام‌متصل، ظرفیت کافی برای یادگیری ترکیبات پیچیده ویژگی‌ها را فراهم می‌کند. این تعداد نه‌چندان کم است که قدرت طبقه‌بندی را محدود کند و نه‌چندان زیاد که باعث بیش‌برازش شود. با توجه به تعداد کلاس‌ها (29 کلاس)، این تعداد نورون انتخاب مناسبی است.

## Dropout با نرخ 0.5

با توجه به تعداد بالای پارامترها در لایه FC1 (حدود 19 میلیون)، از چندین روش منظم‌سازی برای جلوگیری از بیش‌برازش استفاده شده است. Dropout با نرخ 0.5 به‌عنوان یکی از این روش‌ها، به‌خوبی خطر بیش‌برازش را کاهش می‌دهد و در عین حال اجازه می‌دهد مدل اطلاعات مفید را یاد بگیرد. همچنین استفاده از Data Augmentation و کاهش نرخ یادگیری در زمان مناسب، از دیگر اقداماتی است که به پایداری و تعمیم‌پذیری مدل کمک کرده است.

## لایه خروجی 29 نورون

تعداد نورون‌های لایه خروجی دقیقاً برابر با تعداد کلاس‌های مسئله (29 کلاس: A-Z, del, nothing, space) است. هر نورون نشان‌دهنده احتمال تعلق تصویر ورودی به یکی از کلاس‌ها است و خروجی نهایی با تابع Softmax به توزیع احتمال تبدیل می‌شود.



پارامترها در این لایه (حدود 19 میلیون)، این تکنیک به خوبی وابستگی نوروها به یکدیگر را کاهش داده و از بیش‌برازش جلوگیری می‌کند.

۳. **ReduceLROnPlateau**: نرخ یادگیری در صورت توقف بهبود زیان اعتبارسنجی به مدت 3 دوره، به صورت خودکار با ضریب 0.5 کاهش می‌یابد. این کار به مدل کمک می‌کند تا از نقاط بهینه محلی عبور کرده و به نقاط بهینه سراسری نزدیک‌تر شود.

## ۳.۳ پارامترهای آموزش

پارامترهای استفاده‌شده در فرآیند آموزش در جدول زیر خلاصه شده است:

جدول ۲.۳: ابرپارامترهای آموزش

| پارامتر                | مقدار                  |
|------------------------|------------------------|
| تعداد دوره             | 10                     |
| اندازه دسته            | 64                     |
| نرخ یادگیری اولیه      | 0.001                  |
| بهینه‌ساز              | Adam                   |
| تابع زیان              | CrossEntropyLoss       |
| زمان‌بند نرخ یادگیری   | ReduceLROnPlateau      |
| تنظیم زمان‌بند         | patience=3, factor=0.5 |
| معیار ذخیره بهترین مدل | بیشترین دقت اعتبارسنجی |
| فایل وزن‌ها            | models/final_model.pth |

نرخ یادگیری اولیه ۰.۰۰۱ با بهینه‌ساز Adam انتخاب شده است. تابع زیان CrossEntropyLoss برای مسائل طبقه‌بندی چندکلاسه مناسب است و خروجی مدل (لاجیت‌ها) را با برچسب‌های یک‌داغ مقایسه می‌کند. زمان‌بند ReduceLROnPlateau در صورت عدم بهبود زیان اعتبارسنجی به مدت ۳ دوره، نرخ یادگیری را به اندازه ضریب ۵.۰ کاهش می‌دهد. این کار به مدل کمک می‌کند تا در نقاط بهینه محلی گیر نکند و به نقاط بهینه سراسری نزدیک‌تر شود. اندازه دسته ۶۴ با توجه به حافظه GPU و تعادل بین سرعت و پایداری گرادین انتخاب شده است. بهترین مدل بر اساس بیشترین دقت اعتبارسنجی انتخاب و در فایل models/final\_model.pth ذخیره شده است.

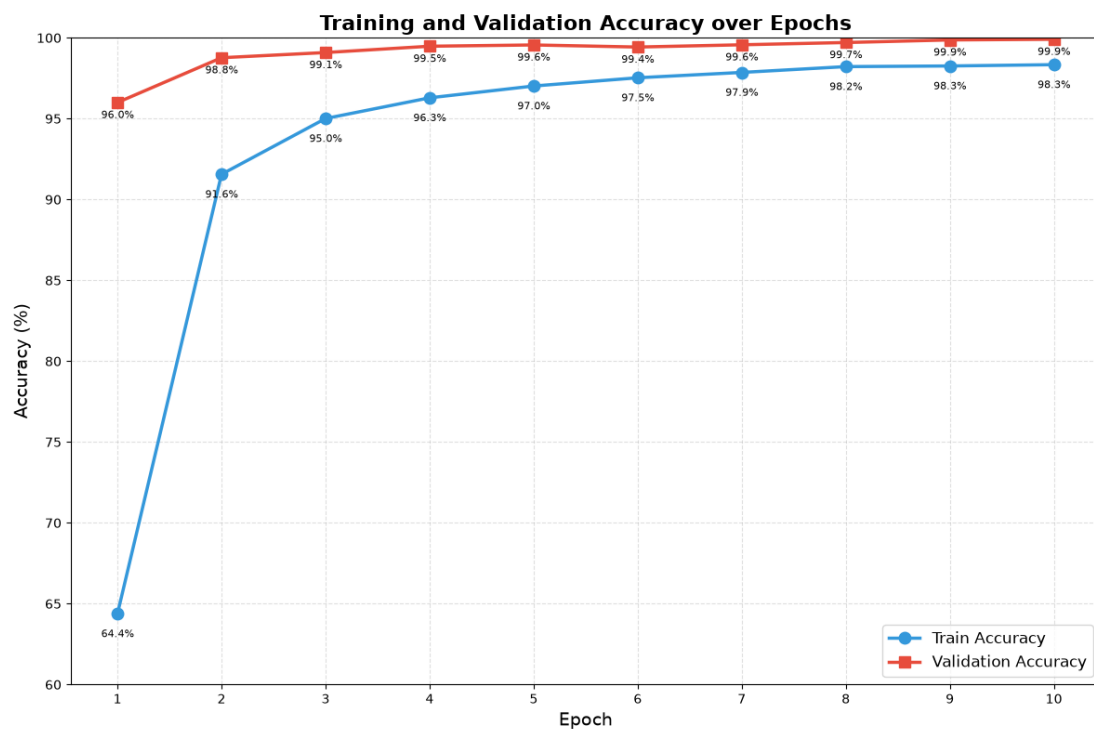
## ۱.۳.۳ انتخاب بهینه‌ساز

بهینه‌ساز Adam به دلیل مزایای زیر برای آموزش مدل انتخاب شده است:

- سرعت همگرایی بالا: Adam با ترکیب Momentum و Adaptive Learning Rate، همگرایی سریع‌تری نسبت به SGD دارد و با تعداد دوره محدود (۱۰ Epoch) به دقت مناسب می‌رسد.
- تنظیم خودکار نرخ یادگیری: این بهینه‌ساز برای هر پارامتر، نرخ یادگیری جداگانه تنظیم می‌کند و نیاز به تنظیم دقیق نرخ یادگیری اولیه را کاهش می‌دهد.
- مقاومت در برابر نویز: داده‌های تصویری شامل نویز هستند و Adam با استفاده از میانگین متحرک گرادیان‌ها، این نویز را کاهش می‌دهد.
- مناسب برای داده‌های بزرگ: با توجه به حجم بالای داده‌های آموزش (۸۷,۰۰۰ تصویر)، Adam عملکرد پایدار و مناسبی ارائه می‌دهد.

اگرچه SGD نیز گزینه‌ای رایج است، اما به دلیل همگرایی کندتر و نیاز به تنظیم دقیق‌تر نرخ یادگیری، برای این پروژه با محدودیت تعداد دوره، انتخاب مناسبی نبود. تجربه عملی نیز نشان داده است که Adam در اکثر پروژه‌های بینایی ماشین به نتایج بهتری می‌رسد.

## ۴.۳ روند آموزش



شکل ۲.۳: دقت آموزش و اعتبارسنجی

جدول ۳.۳: دقت ثبت شده در هر دوره

| دوره | آموزش  | اعتبارسنجی | دوره | آموزش  | اعتبارسنجی |
|------|--------|------------|------|--------|------------|
| ۱    | 64.38% | 96.00%     | ۶    | 97.53% | 99.43%     |
| ۲    | 91.56% | 98.77%     | ۷    | 97.86% | 99.57%     |
| ۳    | 95.01% | 99.09%     | ۸    | 98.22% | 99.71%     |
| ۴    | 96.29% | 99.48%     | ۹    | 98.26% | 99.87%     |
| ۵    | 97.02% | 99.56%     | ۱۰   | 98.34% | 99.91%     |

در ابتدا، فرآیند آموزش با 30 دوره برنامه ریزی شده بود. اما با بررسی روند تغییرات دقت و زیان در دوره های ابتدایی، مشخص شد که مدل به سرعت به دقت بالایی روی داده های اعتبارسنجی دست می یابد و از دوره دوم به بعد، دقت اعتبارسنجی به بیش از 98 درصد می رسد. همچنین از دوره سوم به بعد، دقت اعتبارسنجی همواره بالای 99 درصد بوده و در دوره نهم و دهم به ترتیب به 99.87 و 99.91 درصد رسیده است. دقت آموزش نیز به تدریج از 64.38 درصد در دوره اول به 98.34 درصد در دوره دهم افزایش یافته است.

با توجه به رشد سریع دقت در دوره های ابتدایی و تثبیت آن در محدوده 99 درصد، تعداد دوره ها از 30 به 10 کاهش یافت تا از اتلاف وقت و افزایش ریسک بیش برآزش جلوگیری شود. انتخاب 10 دوره به عنوان تعداد نهایی دوره ها، تعادل مناسبی بین دستیابی به دقت بالا و جلوگیری از بیش برآزش ایجاد می کند.

همان طور که در جدول و نمودار مشاهده می شود، بهترین دقت اعتبارسنجی در دوره دهم با مقدار 99.91 درصد ثبت شده است. بنابراین وزن های مربوط به این دوره به عنوان بهترین مدل ذخیره شده و در ادامه مراحل استنتاج و ارزیابی نهایی مورد استفاده قرار گرفته است. عملکرد مدل در دوره های پایانی نشان دهنده پایداری مناسب مدل و تأیید انتخاب 10 دوره به عنوان تعداد مناسب است.

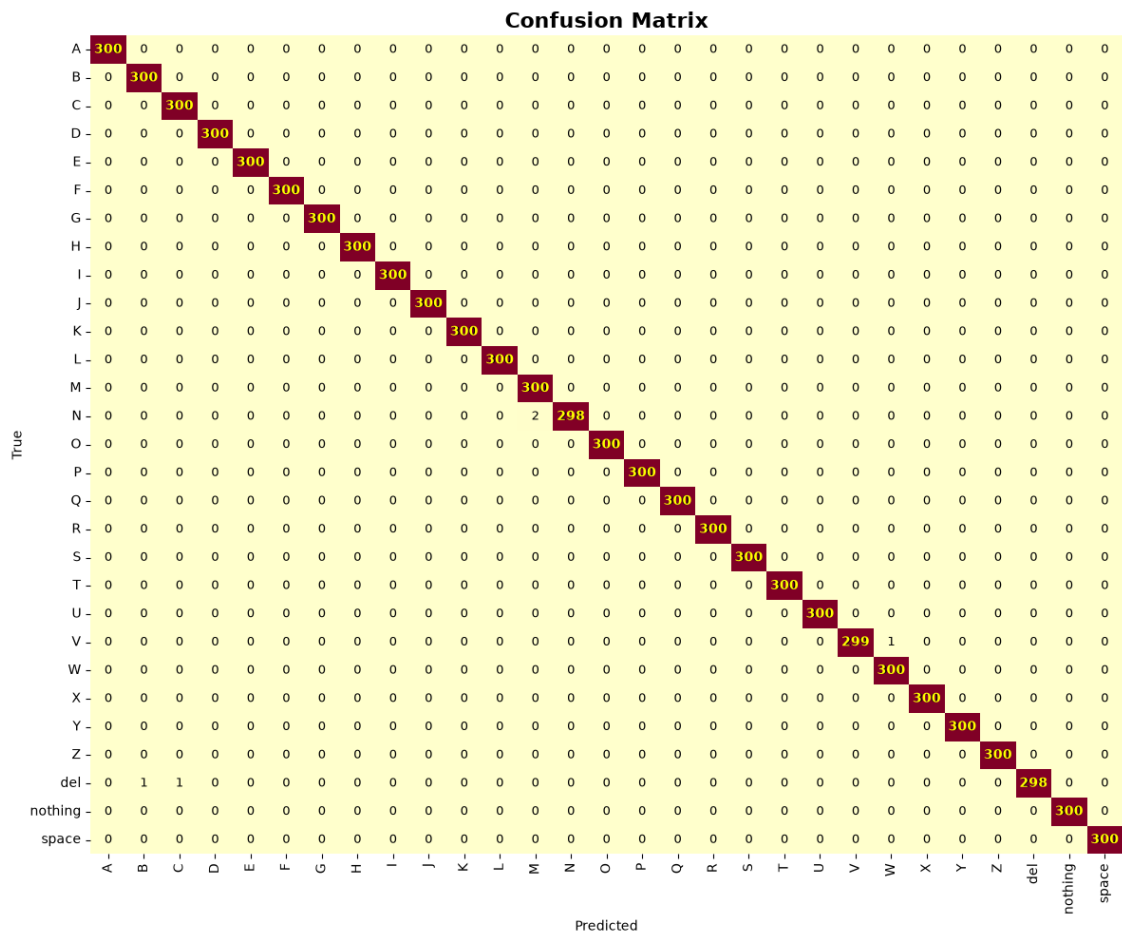
## ۵.۳ ارزیابی نهایی

نتایج ثبت شده روی مجموعه آزمون 8700 تصویری عبارت اند از:

جدول ۴.۳: نتایج نهایی مدل

| معیار                    | مقدار         |
|--------------------------|---------------|
| پیش بینی صحیح            | 8,695 / 8,700 |
| دقت آزمون                | 99.94%        |
| میانگین اطمینان          | 99.89%        |
| کمینه اطمینان مشاهده شده | 40.62%        |

دقت 99.94 درصد روی مجموعه آزمون نشان‌دهنده عملکرد بسیار خوب مدل در تشخیص حروف اشاره است. این نتیجه حاکی از آن است که مدل الگوهای موجود در داده‌های آموزشی را به‌خوبی یاد گرفته و توانسته است روی داده‌های دیده‌نشده نیز تعمیم‌پذیری بالایی داشته باشد. میانگین اطمینان 99.89 درصد نیز بیانگر آن است که مدل در اکثر پیش‌بینی‌ها با اطمینان بالا تصمیم‌گیری می‌کند. کمینه اطمینان مشاهده‌شده (40.62 درصد) مربوط به تعداد بسیار کمی از تصاویر با کیفیت پایین یا نویز بالا است که در ارزیابی کلی تأثیر قابل‌توجهی ندارد.



شکل ۳.۳: ماتریس درهم‌ریختگی روی مجموعه آزمون

ماتریس درهم‌ریختگی نشان می‌دهد که مدل در اکثر کلاس‌ها عملکرد عالی داشته و تعداد خطاها بسیار محدود است. بیشترین خطاها مربوط به کلاس‌های مشابه زیر است:

جدول ۵.۳: پنج جفت کلاس با بیشترین خطا

| کلاس واقعی | پیش‌بینی اشتباه | تعداد خطا |
|------------|-----------------|-----------|
| N          | M               | ۲         |
| V          | W               | ۱         |
| del        | B               | ۱         |
| del        | C               | ۱         |

این خطاها عمدتاً بین کلاس‌هایی رخ داده است که از نظر شکل دست و موقعیت انگشتان شباهت زیادی به یکدیگر دارند. برای مثال، حروف  $N$  و  $M$  در زبان اشاره تنها در تعداد انگشتان باز شده تفاوت دارند و این شباهت بصری باعث سردرگمی مدل شده است. همچنین حروف  $V$  و  $W$  نیز از نظر زاویه انگشتان بسیار نزدیک به هم هستند. مجموع خطاها 5 مورد از 8700 تصویر (معادل 0.06 درصد) است که نشان‌دهنده عملکرد بسیار دقیق مدل است.

## ۶.۳ بهبود مدل با Weighted Loss (تنظیم دقیق)

پس از ارزیابی مدل روی داده‌های تست، عملکرد بسیار مطلوبی با دقت 99.34٪ به دست آمد. با این حال، پس از اجرای مدل در محیط واقعی (فاز سوم) و تست روی تصاویر وب‌کم، مشاهده شد که مدل روی برخی کلاس‌ها عملکرد ضعیف‌تری دارد و آنها را با کلاس‌های مشابه اشتباه می‌گیرد. این کلاس‌ها عمدتاً شامل حروفی هستند که از نظر شکل دست و حالت انگشتان شباهت زیادی به یکدیگر دارند. برای رفع این مشکل، از تکنیک Weighted Loss استفاده شد تا مدل توجه بیشتری به کلاس‌های ضعیف داشته باشد و بتواند تفاوت‌های ظریف بین آنها را بهتر یاد بگیرد.

### ۱.۶.۳ مرحله ۱: وزن‌دهی به کلاس‌های $A$ ، $M$ و $N$

کلاس‌های  $A$ ،  $M$  و  $N$  از جمله کلاس‌هایی بودند که در تشخیص Real-time بیشترین خطا را داشتند. هر سه این حروف در زبان اشاره به صورت مشت با تغییرات جزئی در موقعیت شست و انگشتان اجرا می‌شوند که تشخیص آنها را دشوار می‌کند. به همین دلیل، وزن این کلاس‌ها در تابع هزینه به مقدار 2.5 افزایش یافت. سپس مدل با این وزن‌های جدید به مدت 5 دوره (Epoch) با نرخ یادگیری 0.0001 تنظیم دقیق شد. پس از این مرحله، دقت مدل روی این سه کلاس به طور محسوسی بهبود یافت و خطای تشخیص آنها کاهش پیدا کرد.

### ۲.۶.۳ مرحله ۲: وزن‌دهی به کلاس‌های $A$ ، $S$ و $T$

در مرحله بعد، کلاس‌های  $A$ ،  $S$  و  $T$  مورد بررسی قرار گرفتند. این کلاس‌ها نیز به دلیل شباهت در حالت مشت، در محیط Real-time با چالش مواجه بودند. به‌ویژه کلاس  $S$  که تشخیص آن در شرایط نوری مختلف با دشواری همراه بود. بنابراین، وزن این کلاس‌ها نیز به مقدار 2.5 تنظیم شد. این مرحله با 3 دوره تنظیم دقیق انجام شد تا مدل تفاوت‌های ظریف بین این کلاس‌ها را بهتر تشخیص دهد. نتایج نشان داد که دقت تشخیص این کلاس‌ها به‌ویژه  $S$  و  $T$  در تصاویر Real-time بهبود یافته است.



### ۳.۶.۳ مرحله ۳: یکسان سازی وزن ها

پس از بهبود کلاس های هدف، برای جلوگیری از بیش برآزش و حفظ تعادل مدل، وزن های همه کلاس ها به مقدار 1 برگردانده شد. این کار باعث شد مدل بدون اولویت دهی خاص، روی همه کلاس ها به صورت یکسان تمرکز کند و عملکرد کلی آن پایدار بماند. در نهایت، مدل با وزن های یکسان به مدت 3 دوره دیگر تنظیم دقیق شد تا وزن های نهایی تثبیت شوند و مدل برای استفاده در محیط Real-time آماده گردد.

### ۴.۶.۳ نتایج حاصل از تنظیم دقیق

جدول زیر خلاصه ای از مراحل انجام شده و نتایج حاصل را نشان می دهد:

جدول ۴.۳: خلاصه مراحل ۰۰۰۰۰۰-۰۰۰۰ و نتایج

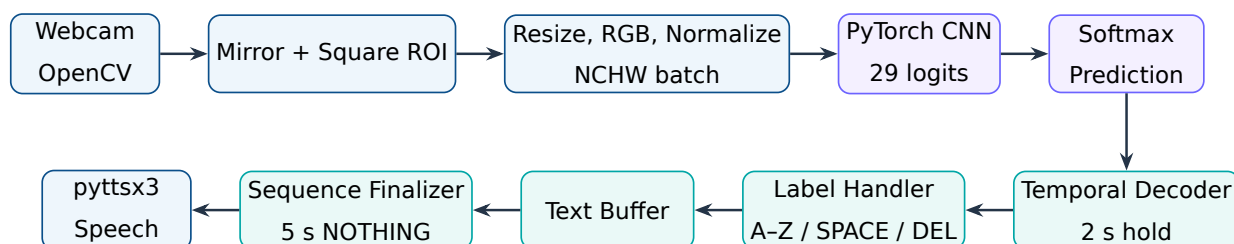
| مرحله   | کلاس های هدف            | تعداد دوره | نتیجه                                  |
|---------|-------------------------|------------|----------------------------------------|
| مرحله ۱ | A, M, N                 | 5          | بهبود تشخیص این کلاس ها                |
| مرحله ۲ | A, S, T                 | 3          | بهبود تشخیص S و T در ۰۰۰۰-۰۰۰۰         |
| مرحله ۳ | همه کلاس ها (وزن یکسان) | 3          | تثبیت عملکرد و جلوگیری از ۰۰۰۰۰۰۰۰۰۰۰۰ |

در نهایت، با استفاده از این رویکرد گام به گام، مدل توانست بدون کاهش دقت روی کلاس های قوی، عملکرد خود را روی کلاس های ضعیف به طور قابل توجهی بهبود بخشد و برای استفاده در محیط واقعی آماده شود. وزن های نهایی مدل در فایل `models/final_model.pth` ذخیره شده است.

## فاز سوم: سامانه بلادرنگ ترجمه به متن و گفتار

### ۱.۴ معماری کلان

کد فاز سوم به صورت یک بسته ماژولار با چهار حوزه اصلی طراحی شده است: بینایی ماشین، استنتاج مدل، رمزگشایی زمانی و گفتار. جریان کامل داده در شکل ۱.۴ آمده است.



شکل ۱.۴: زنجیره پردازش بلادرنگ از وب کم تا گفتار

### ۲.۴ ساختار ماژول ها

```

src/sign_translator/
|-- app.py           # orchestration and main loop
|-- config.py        # camera/model/decoder/TTS parameters
|-- labels.py        # canonical label order and commands
|-- vision/
|   |-- camera.py    # safe camera lifecycle
|   |-- roi.py       # square region of interest
|   |-- overlay.py   # prediction and progress UI
|-- inference/
|   |-- preprocessing.py # BGR -> normalized NCHW
|   |-- torch_model.py  # SignLanguageCNN architecture

```

```
| |-- torch_predictor.py # weight loading and inference
| |-- output_decoder.py # logits/probabilities validation
| `-- factory.py          # mock or torch backend
|-- decoding/
| |-- temporal_decoder.py
| |-- sequence_finalizer.py
| |-- label_handler.py
| `-- text_buffer.py
`-- speech/
    |-- factory.py
    |-- noop.py
    `-- pytttsx3_engine.py
```

این جداسازی باعث می‌شود مدل، موتور گفتار و منطق رابط بدون بازنویسی حلقه اصلی قابل تعویض باشند. برای مثال، TimedMockPredictor پیش از تحویل مدل واقعی امکان تست کل سامانه را فراهم کرده و سپس TorchPredictor از طریق کارخانه جایگزین آن شده است.

## ۳.۴ دریافت تصویر و ناحیه مورد توجه

کلاس Camera دوربین را به صورت context manager باز و در پایان آزاد می‌کند. رزولوشن در تنظیمات روی  $1280 \times 720$  قرار گرفته و تصویر برای تجربه طبیعی کاربر به صورت افقی آینه می‌شود. ناحیه دست با نسبت‌های مستقل از رزولوشن تعریف شده است:

$$\begin{aligned}x &= 0.55W, & y &= 0.12H, \\w &= 0.38W, & h &= 0.72H.\end{aligned}$$

سپس ضلع کوچک‌تر انتخاب و کادر دقیقاً مربعی می‌شود. این اقدام از فشردگی یا کشیدگی شکل دست هنگام تبدیل به ورودی  $200 \times 200$  جلوگیری می‌کند.

## ۴.۴ همسان‌سازی پیش‌پردازش آموزش و اجرا

فریم OpenCV در قالب BGR است، در حالی که داده آموزشی RGB بوده است. مسیر استنتاج مراحل زیر را انجام می‌دهد:

۱. اعتبارسنجی ورودی سه کاناله و غیرخالی

۲. تغییر اندازه به  $200 \times 200$

۳. تبدیل BGR به RGB

۴. تبدیل نوع به float32 و تقسیم بر 255

۵. نرمال سازی کانالی با میانگین و انحراف معیار آموزش

۶. تبدیل چیدمان از HWC به CHW

۷. افزودن بعد دسته و تولید تنسور  $1 \times 3 \times 200 \times 200$

انتخاب روش درون یابی نیز بر اساس کوچک سازی یا بزرگ سازی انجام می شود: INTER\_AREA برای کاهش اندازه و INTER\_LINEAR برای افزایش اندازه.

## ۵.۴ بارگذاری مدل و استنتاج

TorchPredictor معماری را با تعداد خروجی برابر طول فهرست برچسب ها می سازد و فایل وزن را با `weights_` `only=True` بارگذاری می کند. استفاده از `strict=True` در `load_state_dict` سبب می شود هر ناسازگاری میان معماری و وزن ها به جای تولید نتیجه اشتباه، با خطای واضح متوقف شود. ترتیب خروجی مدل در کد به صورت زیر ثابت شده است:

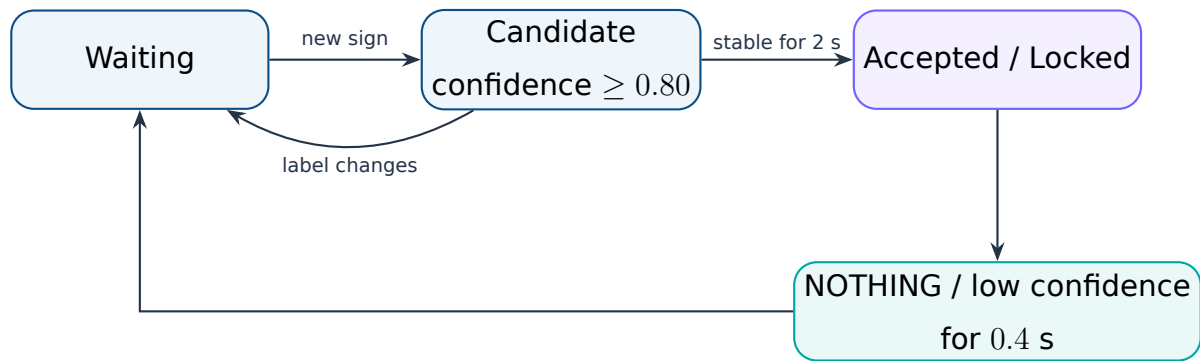
(A, ..., Z, DEL, NOTHING, SPACE)

خروجی شبکه لاجیت است. `output_decoder.py` شکل خروجی، تعداد کلاس ها، مقادیر نامتناهی و تکراری نبودن برچسب ها را بررسی و سپس softmax پایدار عددی را محاسبه می کند.

## ۶.۴ منطق تثبیت زمانی

پیش بینی خام هر فریم مستقیماً وارد متن نمی شود. TemporalDecoder یک ماشین حالت ساده دارد:

- ▶ پیش بینی باید اطمینان حداقل 0.80 داشته باشد
- ▶ یک برچسب باید به مدت 2.0 s بدون تغییر باقی بماند
- ▶ پس از ثبت، همان برچسب قفل می شود تا با نگه داشتن دست چندبار تکرار نشود
- ▶ نمایش NOTHING یا افت اطمینان به مدت 0.4 s قفل را آزاد می کند
- ▶ برای تکرار دو حرف یکسان، مانند OO، کاربر باید بین دو نمایش، حالت NOTHING را نشان دهد



شکل ۲.۴: ماشین حالت رمزگشایی زمانی

## ۲.۴ تفسیر برچسب‌ها و بافر متن

جدول ۱.۴: رفتار هر نوع برچسب پذیرفته‌شده

| برچسب   | عملکرد                                                       |
|---------|--------------------------------------------------------------|
| A-Z     | افزودن حرف انگلیسی بزرگ به انتهای بافر                       |
| SPACE   | افزودن یک فاصله؛ فاصله ابتدایی یا تکراری نادیده گرفته می‌شود |
| DEL     | حذف آخرین کاراکتر، شامل حرف یا فاصله                         |
| NOTHING | عدم تغییر متن؛ استفاده به عنوان علامت خنثی و پایان رشته      |

TextBuffer به جای ذخیره توکن‌های کلمه، فهرست کاراکترها را نگه می‌دارد. بنابراین فرمان حذف دقیقاً آخرین کاراکتر را برمی‌گرداند و متن نهایی با حذف فاصله‌های ابتدا و انتها تولید می‌شود.

## ۸.۴ تشخیص پایان عبارت و تبدیل به گفتار

SequenceFinalizer تنها زمانی شمارنده را فعال می‌کند که:

► برچسب NOTHING با اطمینان حداقل 0.80 تشخیص داده شود.

► بافر متن خالی نباشد.

► این حالت برای 5.0 s پیوسته ادامه یابد.

پس از تکمیل شمارنده، متن یکبار نهایی می‌شود، به موتور گفتار ارسال و سپس بافر پاک می‌شود. پرچم داخلی از چندبارخواندن متن در همان دوره طولانی NOTHING جلوگیری می‌کند.

موتور pyttsx3 در یک نخ کارگر اجرا می شود و متن ها را از صف دریافت می کند. این طراحی مانع توقف حلقه پردازش ویدئو هنگام پخش صدا می شود. یک پیاده سازی NoOp نیز برای تست و محیط های فاقد صوت وجود دارد.

## ۹.۴ رابط تصویری

overlay.py اطلاعات زیر را روی فریم نشان می دهد:

- ▶ کادر سبز ناحیه دست
- ▶ برچسب پیش بینی شده و درصد اطمینان
- ▶ درصد پیشرفت نگه داشتن علامت
- ▶ وضعیت قفل، پذیرش یا آزاد شدن علامت
- ▶ درصد پیشرفت نهایی سازی متن
- ▶ متن جاری و آخرین متن نهایی شده
- ▶ راهنمای خروج با کلیدهای Q یا ESC

## اعتبارسنجی نرم افزار و کیفیت پیاده سازی

### ۱.۵ مجموعه تست ها

پوشه tests اجزای زیر را پوشش می دهد:

جدول ۱.۵: حوزه های تحت آزمون واحد

| فایل تست                   | رفتارهای بررسی شده                                              |
|----------------------------|-----------------------------------------------------------------|
| test_roi.py                | محاسبه کادر، برش تصویر و مربعی بودن ناحیه ورودی                 |
| test_preprocessing.py      | شکل NCHW، تبدیل رنگ، نوع داده و نرمال سازی کانال ها             |
| test_torch_model.py        | تولید ۲۹ لاجیت برای دسته ورودی 200×200                          |
| test_torch_predictor.py    | اتصال پیش پردازش به مدل ساختگی، نگاشت خروجی و مدیریت فایل مفقود |
| test_output_decoder.py     | لاجیت، احتمال، softmax، تعداد کلاس و مقادیر نامعتبر             |
| test_temporal_decoder.py   | شرط دوثانیه ای، جلوگیری از تکرار، آزاد شدن و آستانه اطمینان     |
| test_sequence_finalizer.py | شرط پنج ثانیه ای، عدم نهایی سازی تکراری و بافر خالی             |
| test_label_handler.py      | رفتار حروف، فاصله، حذف، NOTHING و برچسب ناشناخته                |
| test_text_buffer.py        | افزودن، حذف، فاصله، پاک سازی و متن نهایی                        |
| test_mock_predictor.py     | صحت خروجی پیش بینی کننده ساختگی                                 |

تیم پروژه اجرای موفق تمام تست های موجود و بدون خطای بررسی Ruff را در محیط توسعه گزارش کرده است. تنظیمات Ruff نوت بوک ها را از تحلیل کد پایتون حذف می کند، زیرا خروجی و ساختار سلولی نوت بوک الزاماً قواعد یک ماژول تولیدی را رعایت نمی کند.

## ۲.۵ مدیریت خطا

موارد زیر با خطاهای اختصاصی یا پیام‌های واضح کنترل شده‌اند:

- ▶ بازنشدن دوربین یا شکست در خواندن فریم
- ▶ نبودن فایل مدل یا ناسازگاری وزن‌ها با معماری
- ▶ درخواست CUDA در سیستم فاقد CUDA
- ▶ تصویر خالی، تعداد کانال اشتباه یا آمار نرمال‌سازی نامعتبر
- ▶ خروجی مدل با شکل نامعتبر، تعداد کلاس اشتباه یا مقادیر NaN/Inf
- ▶ برچسب خالی، تکراری یا ناشناخته
- ▶ زمان‌های غیرصعودی در ماشین‌های حالت

## ۳.۵ قابلیت تعویض اجزا

دو کارخانه مجزا برای پیش‌بینی و گفتار وجود دارد. در مسیر پیش‌بینی، حالت mock برای توسعه و حالت torch برای مدل نهایی انتخاب می‌شود. در مسیر صوت نیز noop و pytt3 قابل انتخاب هستند. این الگو تست‌پذیری را افزایش و وابستگی مستقیم حلقه اصلی به کتابخانه‌ها را کاهش داده است.



## تحلیل محدودیت‌ها و پیشنهادهای بهبود

### ۱.۶ شکاف دامنه میان دیتاست و وب کم

داده آموزشی از محیط، زاویه و نور نسبتاً همگنی برخوردار است. عملکرد 99.34% روی تقسیم داخلی همان منبع، دقت روی دوربین و فرد جدید را تضمین نمی‌کند. کلاس‌های بصری نزدیک مانند O/C/Q یا V/U/W معمولاً به زاویه انگشتان و وضوح مرزها حساس‌اند.

### ۲.۶ حروف و حرکات پویا

مدل ورودی تک‌فریم دارد؛ در نتیجه حرکت زمانی را مستقیماً مدل نمی‌کند. برخی حروف زبان اشاره مانند J و Z در اجرای طبیعی شامل مسیر حرکتی‌اند. مدل ممکن است یک وضعیت ثابت از این حرکت را طبقه‌بندی کند، اما برای شناخت صحیح حرکت، استفاده از دنباله فریم و معماری‌هایی مانند 3D-CNN، LSTM یا Transformer مناسب‌تر است.

# راهنمای اجرا و ساختار تحویل

## ۱.۷ نصب

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -e ".[dev,tts,model]"
```

برای نصب PyTorch سازگار با کارت گرافیک، درایور و نسخه CUDA باید از بسته مناسب سیستم استفاده شود. در صورت نبود CUDA، مقدار `device="auto"` به صورت خودکار CPU را انتخاب می کند.

## ۲.۷ اجرای کنترل کیفیت

```
pytest
ruff check .
```

## ۳.۷ اجرای سامانه

فایل وزن باید در مسیر زیر قرار داشته باشد:

`models/final_model.pth`

سپس برنامه از ریشه پروژه اجرا می شود:

```
python main.py
```

## ۴.۷ ساختار فایل نهایی تحویل

طبق دستور پروژه، تمام محتوا باید در یک فایل فشرده با الگوی نام‌گذاری تعیین‌شده قرار گیرد:

```
AI-Project3-StudentID1-StudentID2.zip
|-- main.py
|-- pyproject.toml
|-- models/
|   |-- final_model.pth
|-- notebooks/
|   |-- phase1&2.ipynb
|   |-- dataset_stats.pt
|-- reports/
|   |-- report.pdf
|   |-- report.tex
|-- src/sign_translator/
|-- tests/
|-- README.md
|-- LICENSE
```

# نتیجه گیری

پروژه حاضر سه بخش داده، مدل و محصول بلادرنگ را در یک زنجیره یکپارچه قرار داده است. فاز اول نشان داد داده آموزشی از نظر تعداد کلاس‌ها، اندازه تصویر و حالت رنگ ساختاری منظم دارد. در فاز دوم، شبکه چهاربخشی CNN با افزایش داده و Dropout به دقت آزمون 99.34% رسید. در فاز سوم، مدل از محیط نوت‌بوک خارج و در یک برنامه واقعی مبتنی بر وب‌کم، رمزگشایی زمانی، ویرایش متن و تبدیل به گفتار به کار گرفته شد.

نقطه قوت اصلی پیاده‌سازی، جداسازی مسئولیت‌ها و توسعه افزایشی بر اساس قرارداد پیش‌بینی است. کد وب‌کم و منطق متن پیش از آماده‌شدن مدل با پیش‌بینی‌کننده ساختگی تکمیل شد و پس از تحویل وزن‌های PyTorch، تنها لایه استنتاج جایگزین گردید. استفاده از تست‌های واحد و اعتبارسنجی صریح داده‌ها نیز ریسک خطاهای خاموش در اتصال مدل را کاهش داده است.

## تنظیمات نهایی سامانه

جدول آ.۱: مقادیر پیش فرض تنظیمات اجرایی

| تنظیم             | مقدار                                     |
|-------------------|-------------------------------------------|
| رزولوشن دوربین    | 1280×720                                  |
| آینه سازی         | فعال                                      |
| اندازه ورودی مدل  | 200×200×3                                 |
| ترتیب رنگ         | RGB                                       |
| چیدمان تانسور     | NCHW                                      |
| مدل               | models/final_model.pth                    |
| دستگاه            | auto (در صورت امکان، در غیر این صورت CPU) |
| آستانه ثبت علامت  | 0.80                                      |
| زمان تثبیت علامت  | 2.0 s                                     |
| زمان آزادسازی قفل | 0.4 s                                     |
| زمان نهایی سازی   | 5.0 s                                     |
| موتور گفتار       | pyttsx3                                   |
| کلیدهای خروج      | ESC و Q                                   |

## قرارداد وزن‌های مدل

فایل وزن ذخیره‌شده یک OrderedDict شامل ۱۲ تنسور پارامتر است. ابعاد مشاهده‌شده در آزمون بارگذاری عبارت‌اند از:

|                         |                  |
|-------------------------|------------------|
| conv1.weight            | (32, 3, 3, 3)    |
| conv1.bias              | (32,)            |
| conv2.weight            | (64, 32, 3, 3)   |
| conv2.bias              | (64,)            |
| conv3.weight            | (128, 64, 3, 3)  |
| conv3.bias              | (128,)           |
| conv4.weight            | (256, 128, 3, 3) |
| conv4.bias              | (256,)           |
| fully_connected1.weight | (512, 36864)     |
| fully_connected1.bias   | (512,)           |
| fully_connected2.weight | (29, 512)        |
| fully_connected2.bias   | (29,)            |

این ابعاد با معماری گزارش‌شده و تعداد پارامتر کل مدل سازگارند.